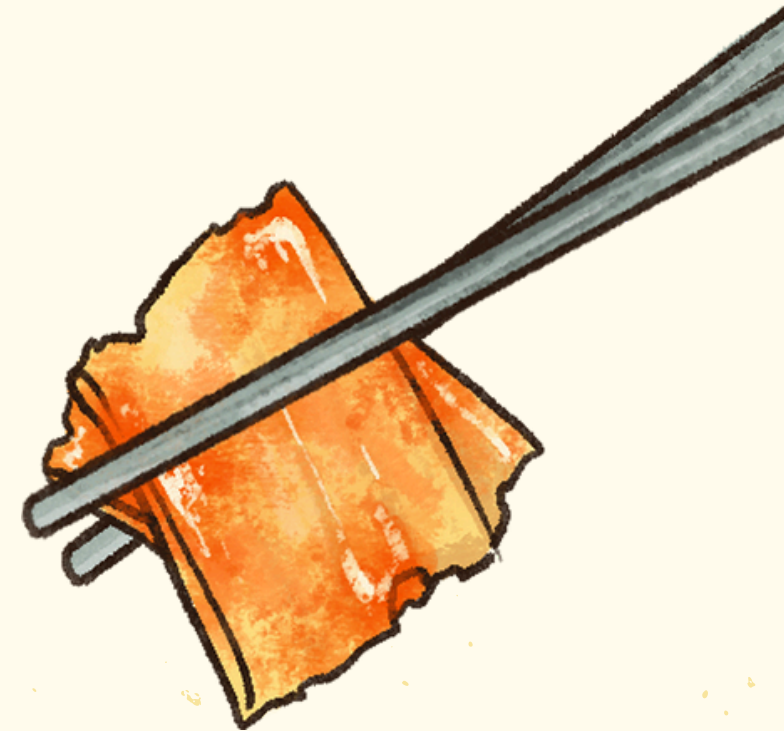


Jantar dos Filósofos



Problema

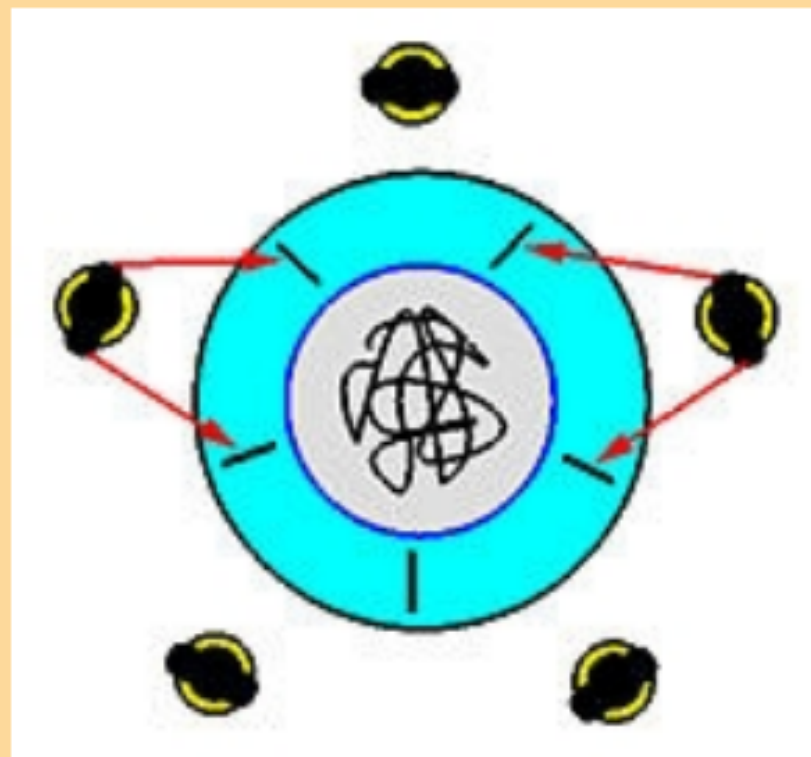
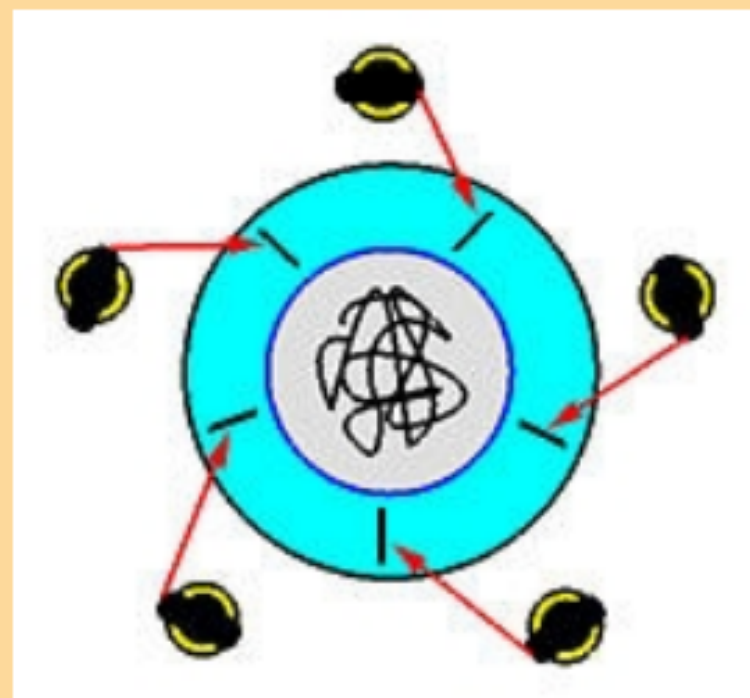
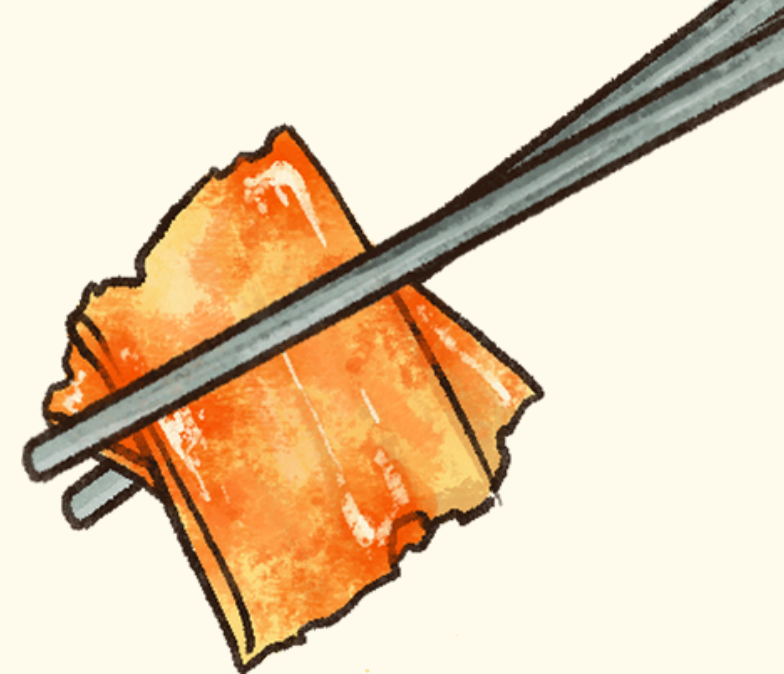


Problema



1. filósofos sentam ao redor de uma mesa redonda.
2. Entre cada par de filósofos existe um hashi (total: 5 hashi).
3. Cada filósofo alterna entre pensar e comer.
4. Para comer, cada filósofo precisa pegar os dois hashi ao seu lado (esquerda e direita)

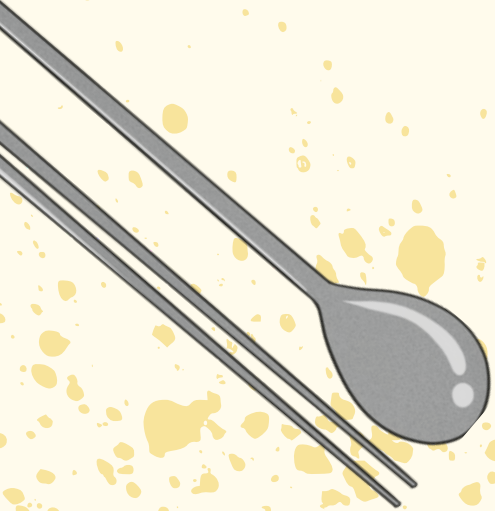
Problema



Se todos tentarem pegar o hashi da esquerda ao mesmo tempo, cada um ficará com apenas 1 hashi e nenhum poderá pegar o segundo → **ocorre deadlock**.

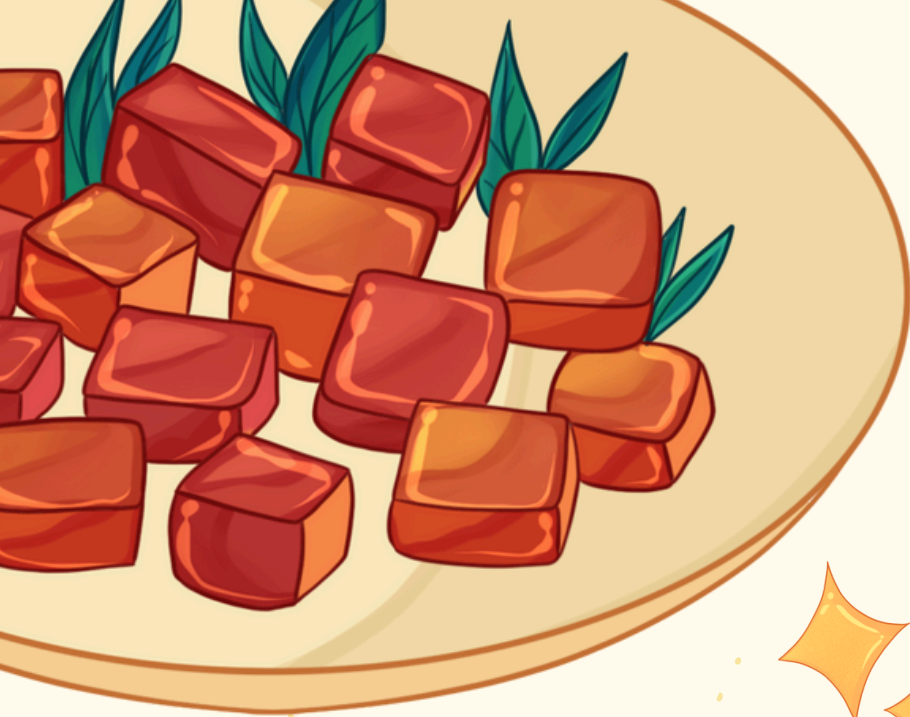
Outros possíveis problemas:

- **Starvation**: um filósofo nunca consegue os dois garfos.
- **Corrida (race conditions)**: múltiplos acessos concorrentes aos mesmos recursos.
- **Exclusão mútua**: um hashi só pode ser usado por 1 filósofo de cada vez.

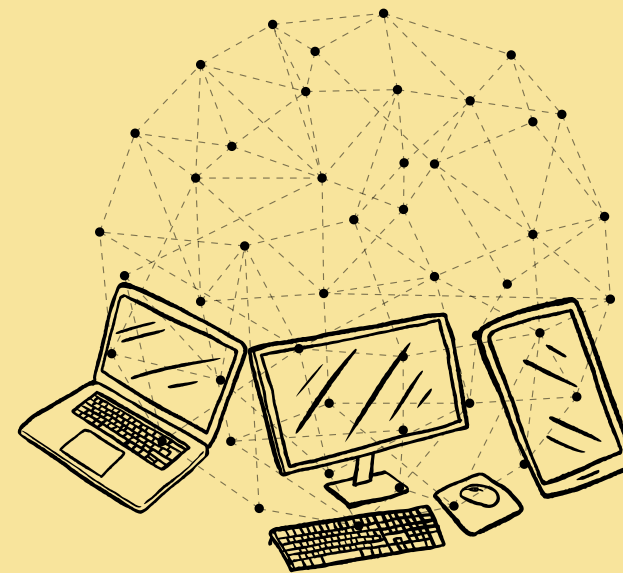


**Por que esse
problema é
importante?**

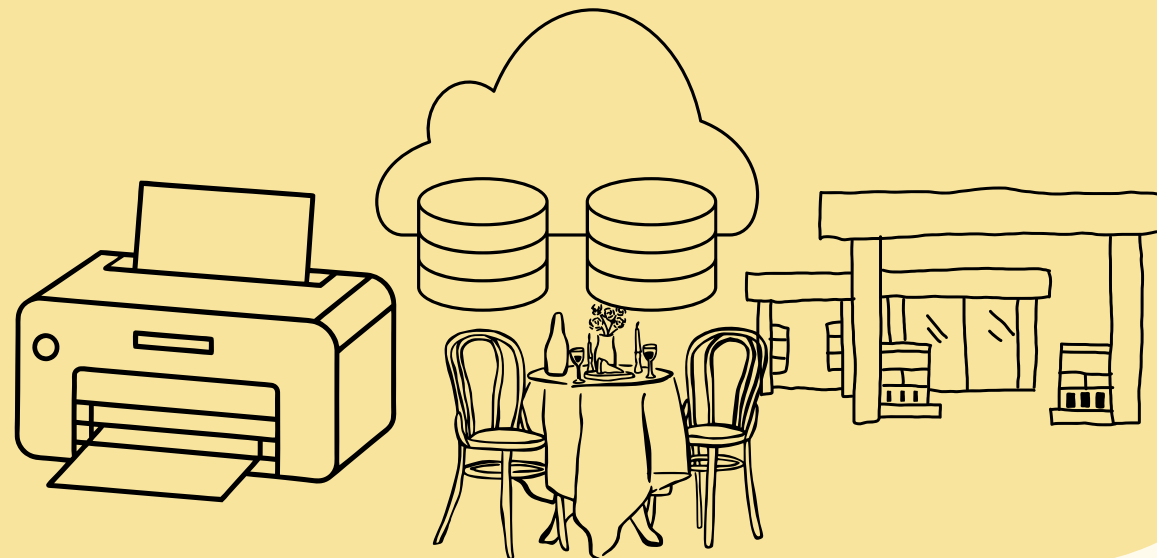




Concorrência e recursos compartilhados são comuns em SO e sistemas distribuídos.



Situações reais: impressoras, bases de dados, vagas em postos, mesas em restaurantes.



O objetivo: projetar controles que evitem falhas sistêmicas (deadlock) sem sacrificar disponibilidade.



Soluções

1. Um garçom (waiter)

Existe um semáforo que limita o número de filósofos tentando comer ao mesmo tempo (máx. 4). Isso evita deadlock.

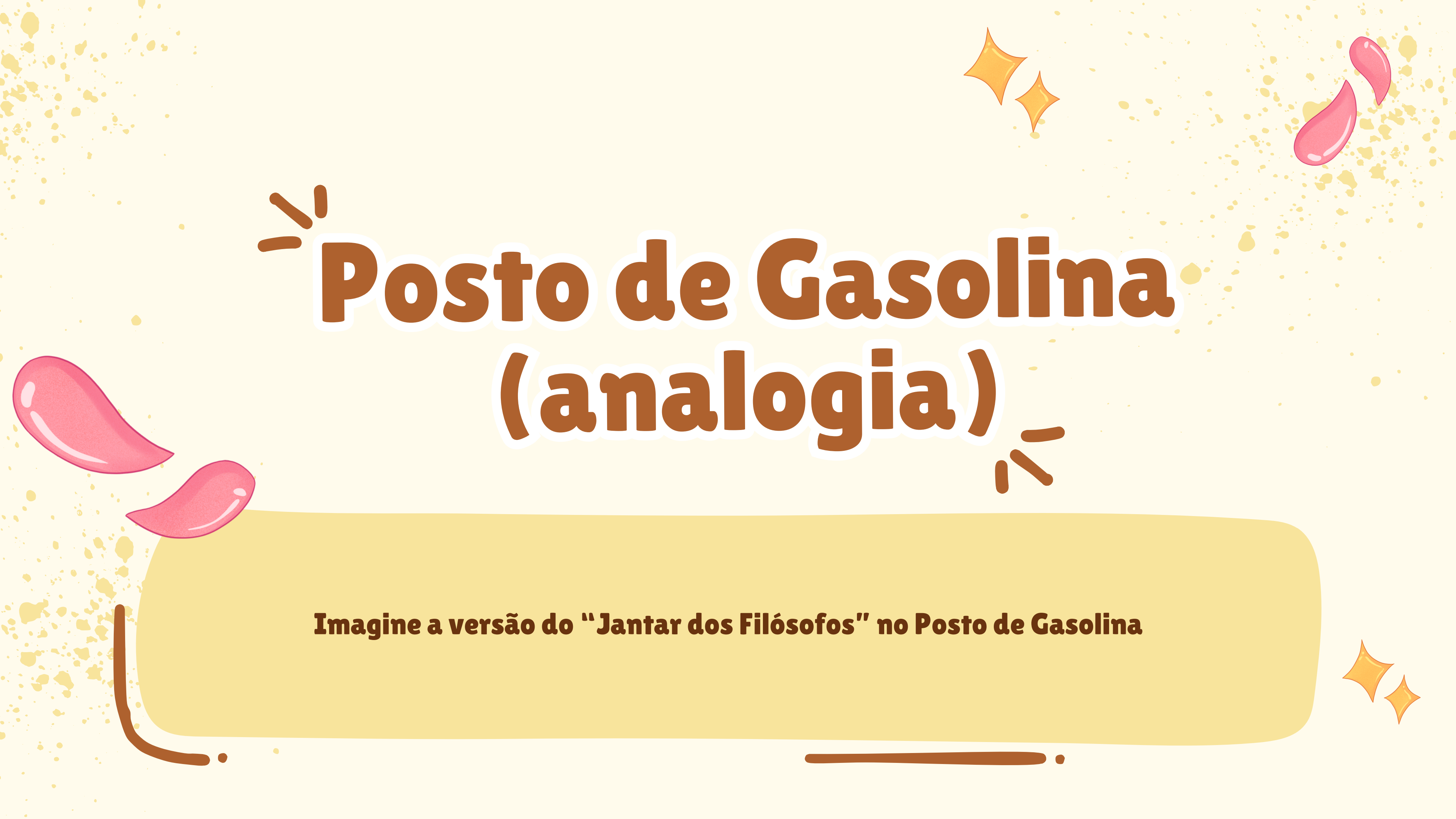
2. Pegar os garfos com ordem definida

Cada filósofo deve pegar sempre primeiro o hashi de menor ID, depois o de maior ID. Isso evita o ciclo circular de espera, que é o que causa o deadlock.

3. Filósofos pares e ímpares pegam hashis em ordem oposta

Alterna a ordem de quem pega hashis, quebrando a simetria.





Posto de Gasolina (analogia)

Imagine a versão do “Jantar dos Filósofos” no Posto de Gasolina



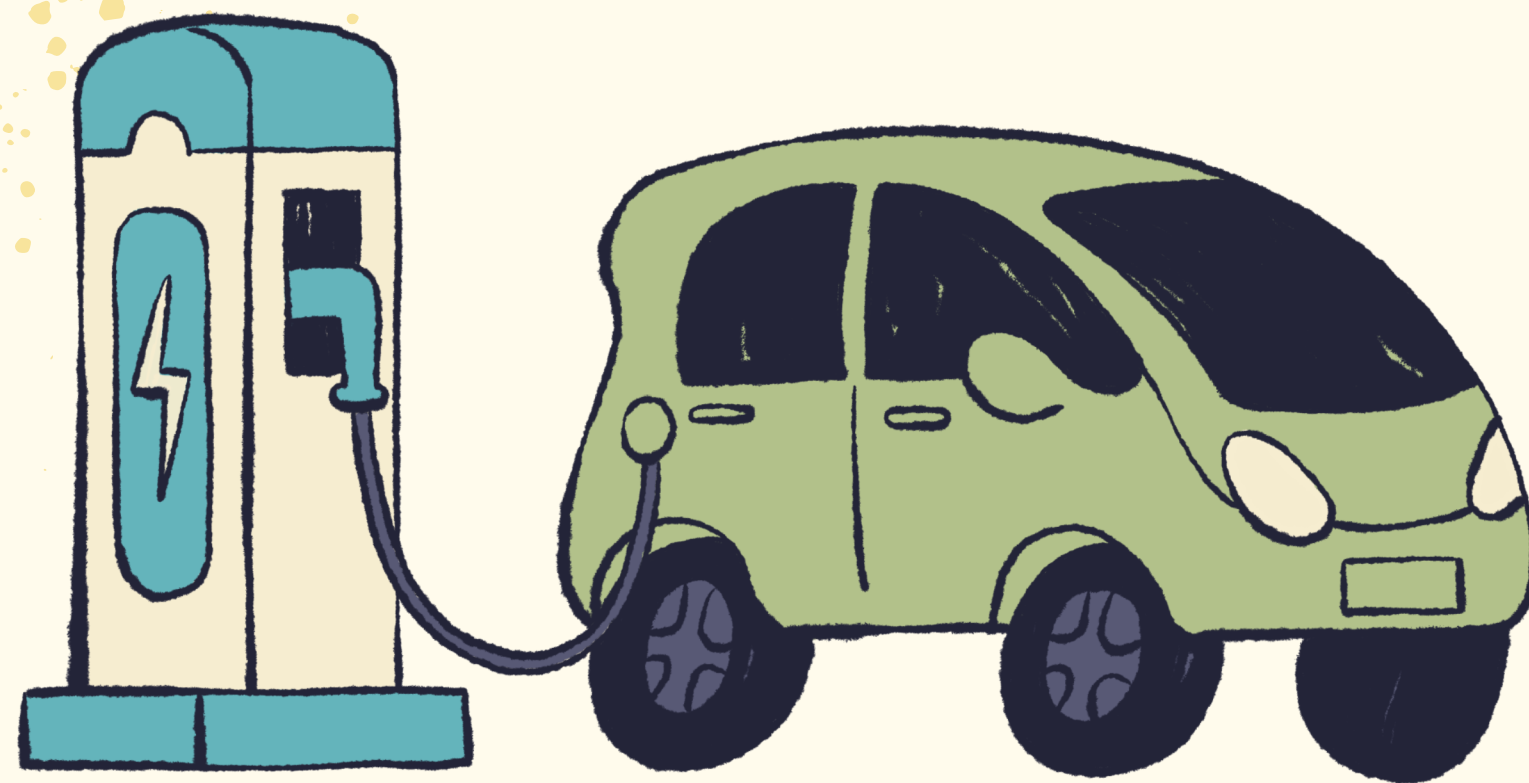
Analogia



- **Existem 5 carros (filósofos).**
- **Existem 5 bombas de combustível (hashis).**
- **Para abastecer, cada carro precisa de duas bombas:**
 - **Uma para gasolina**
 - **Outra para calibrar pneus (ou lavar o para-brisa, etc.)**

As bombas ficam posicionadas em círculo, como os hashis da mesa dos filósofos.

Regra do sistema



Cada carro precisa usar duas bombas ao mesmo tempo. Se um carro pegar somente uma e não conseguir a outra, ele fica travado.

Isso pode gerar o mesmo problema do Jantar dos Filósofos:

✗ Possível Deadlock no Posto

Se todos os carros pegarem a bomba da esquerda primeiro, e ficarem esperando a da direita:

- **Cada carro fica segurando uma bomba**
- **Nenhum consegue pegar a segunda**
- **Ninguém abastece → deadlock**

Solução: Garçon do Posto



1. Colocamos um frentista coordenador (um semáforo).
2. Ele só permite que até 4 carros tentem abastecer ao mesmo tempo. Isso impede que todas as bombas fiquem travadas simultaneamente.

FUNCIONAMENTO

1. O carro pede permissão para o frentista.
2. Se houver vagas (máx. 4 carros usando bombas), ele entra.
3. Só então o carro tenta pegar as duas bombas.
4. Abastece.
5. Devolve as bombas.
6. Avisa o frentista que terminou.



EXECUÇÃO DO CÓDIGO

```
[21:38:00] Controle de Bombas iniciado com 5 bombas. Frentista ativado (N-1).  
[21:38:00] Carro 1 está dirigindo / pensando...  
[21:38:00] Carro 0 está dirigindo / pensando...  
[21:38:00] Carro 2 está dirigindo / pensando...  
[21:38:00] Carro 3 está dirigindo / pensando...  
[21:38:00] Carro 4 está dirigindo / pensando...  
[21:38:01] Carro 4 está pedindo permissão ao frentista...  
[21:38:01] Carro 4 aguardando frentista...  
[21:38:01] Carro 4 Frentista liberou o carro  
[21:38:01] Carro 4 ALOCOU as Bombas 0 e 1  
[21:38:01] Carro 4 conseguiu duas bombas. Iniciando abastecimento...  
[21:38:01] Carro 1 está pedindo permissão ao frentista...  
[21:38:01] Carro 1 aguardando frentista...  
[21:38:01] Carro 1 Frentista liberou o carro  
[21:38:01] Carro 1 ALOCOU as Bombas 2 e 3  
[21:38:01] Carro 1 conseguiu duas bombas. Iniciando abastecimento...
```




**AGRADECEMOS
PELA ATENÇÃO!**

